

Universidad Rafael Landívar
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas.
Lenguajes Formales y Autómatas - Sección: 02
Catedrático: Ing. Vivian Damaris Campos González.



Analizador de operaciones aritméticas

Manual Técnico

Estudiante: Tony Alexander Balán Mendoza.
Carné: 1202124

Gabriel Emilio Toyom Jimenez.
Carné: 1051524

Guatemala, 02 de noviembre de 2025.

ÍNDICE

I.	RESUMEN	1
I.	Descripción del sistema	1
II.	Problema de resolución.....	1
III.	Aspectos generales del sistema	1
II.	ARQUITECTURA DEL SISTEMA	2
I.	Diagrama de clases.....	2
II.	Diagrama de flujo.....	3
III.	Descripción de clases importantes	4
1.	Clase Main:	4
2.	Clase Scanner:	4
3.	Clase Token:.....	5
4.	Clase AER:	6
5.	Clase Sintactico:	6
6.	Clase AES:.....	8
7.	Clase GeneradorArbol:	9
8.	Clase GeneradorHTML:.....	10
9.	Clase InterfazGrafica:.....	11

I. RESUMEN

I. Descripción del sistema

Este sistema es un analizador léxico desarrollado en Java que tiene como propósito principal leer un archivo de entrada, tokenizar el contenido y validar los lexemas según las reglas definidas por el lenguaje fuente.

El sistema está diseñado para funcionar sin el uso de librerías externas de análisis léxico, utilizando expresiones regulares y estructuras propias para el reconocimiento de tokens.

II. Problema de resolución

Se solicita el desarrollo de un programa capaz de leer código fuente en un formato específico. Este programa deberá identificar el lenguaje del código, detectar errores léxicos y ejecutar las instrucciones correspondientes.

III. Aspectos generales del sistema

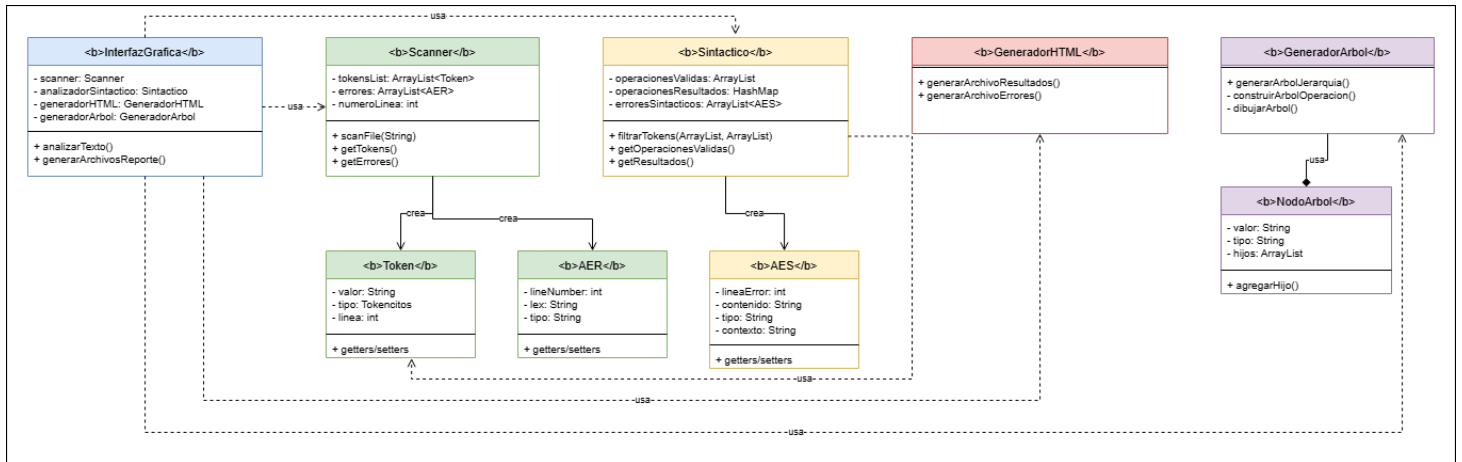
- Lenguaje de programación: Se implementó como lenguaje principal del sistema a Java y HTML para la generación de reportes.
- Entorno de Desarrollo: El código fuente del sistema se desarrolló en el Visual Studio Code. También, se utilizó Git para llevar un control de versiones para gestionar el desarrollo colaborativo y mantener el historial de cambios en el código fuente.
- Archivos de configuración: Se utilizaron archivos .txt para comprobar el funcionamiento de la aplicación y realizar los cambios pertinentes para cumplir con los requerimientos del sistema.

II. ARQUITECTURA DEL SISTEMA

I. Diagrama de clases

Figura 01

Diagrama de clases del proyecto.

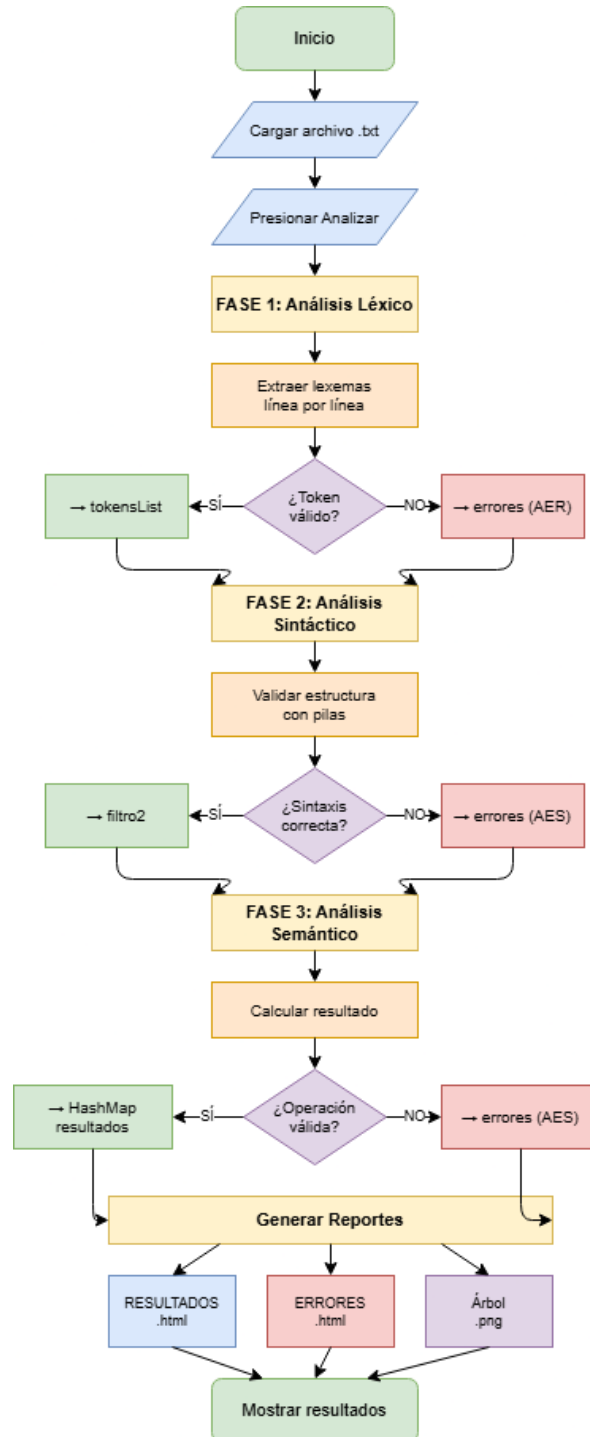


Nota. Fuente: Elaboración propia en Draw.io.

II. Diagrama de flujo

Figura 02

Diagrama del flujo principal del proyecto



Nota. Fuente: Elaboración propia en Draw.io.

III. Descripción de clases importantes

1. Clase Main:

Propósito: Punto de entrada del programa.

Métodos principales:

- `main(String[] args)`: Ejecuta el flujo principal del sistema, incluyendo la lectura del archivo y el análisis léxico.

Notas de lógica interna:

- Crea una instancia del analizador léxico y llama a sus métodos para procesar el archivo.

2. Clase Scanner:

Propósito: La clase Scanner es el núcleo del análisis léxico. Se encarga de leer un archivo línea por línea, extraer lexemas, clasificarlos como tokens válidos o errores léxicos, y almacenarlos para su posterior uso o visualización.

Atributos:

- `ArrayList<Token> tokensList`: Lista de tokens válidos reconocidos.
- `ArrayList<AER> errores`: Lista de errores léxicos encontrados.
- `int numeroLinea`: Contador de líneas del archivo para asociar tokens y errores con su ubicación.

Métodos Principales:

- `scanFile(String fileName)`
 - Función: Lee el archivo línea por línea y llama a `procesarLinea()` para analizar cada una.
 - Manejo de errores: Captura excepciones por archivo no encontrado o problemas de lectura.
- `procesarLinea(String linea)`
 - Función: Limpia la línea, extrae lexemas y los analiza individualmente con `verificacionLineas()`.
- `extraerLexemas(String linea)`
 - Función: Divide la línea en lexemas, respetando etiquetas `<...>` como unidades indivisibles.
 - Lógica especial: Usa un `StringBuilder` para acumular caracteres y detectar etiquetas correctamente.
- `verificacionLineas(String lexema)`

- Función: Clasifica el lexema como etiqueta de apertura/cierre, número o error.
- Lógica: Llama a verificarApertura() o verificarCierre() según el tipo de etiqueta.
- verificarApertura(String lexema)
 - Función: Valida etiquetas de apertura como <Operacion=SUMA>, <Numero>, <P>, <R>.
 - Lógica interna: Extrae el operador y lo valida con verificarOperacion().
- verificarCierre(String lexema)
 - Función: Valida etiquetas de cierre como </Operacion>, </Numero>, etc.
 - Lógica interna: Extrae el contenido y lo compara con etiquetas válidas.
- verificarOperacion(String op)
 - Función: Verifica si el operador es uno de los permitidos (SUMA, RESTA, MULTIPLICACION, etc.).
- esNumero(String str)
 - Función: Valida si un lexema representa un número (entero o decimal).
 - Lógica: Verifica formato correcto, presencia de dígitos y uso adecuado del punto decimal.
- mostrarTokens()
 - Función: Imprime todos los tokens reconocidos.
- mostrarErrores()
 - Función: Imprime todos los errores léxicos encontrados.

Notas de lógica interna:

- El sistema distingue etiquetas de apertura y cierre usando los caracteres < y >.
- Las etiquetas de apertura pueden contener parámetros como Operacion=SUMA.
- Se valida que las etiquetas no estén vacías y que los operadores sean conocidos.
- Los números se validan manualmente sin expresiones regulares.
- Los errores se registran con línea, lexema y descripción.

3. Clase Token:

Propósito: Representa un token válido del lenguaje.

Atributos:

- String tipo: Tipo del token (e.g., IDENTIFICADOR, NUMERO).
- String valor: Valor textual del token.
- int linea: Línea del archivo donde se encontró.

Métodos principales:

- Constructor: Inicializa los atributos.
- toString(): Devuelve una representación legible del token.

Notas de lógica interna: Se usa para encapsular la información de cada lexema válido.

4. Clase AER:

Propósito: La clase AER (Almacena Error de Reconocimiento) representa un error léxico detectado durante el análisis de un archivo fuente. Su función principal es encapsular la información relevante sobre el error para su posterior reporte.

Atributos:

- int lineNumber: Número de línea donde ocurrió el error.
- String lex: Lexema que causó el error.
- String tipo: Descripción del tipo de error (por ejemplo, "Etiqueta vacía", "Lexema no reconocido").

Métodos principales:

- AER(int numLinea, String lexito, String tokensito)
 - Constructor: Inicializa un objeto AER con la línea, el lexema y el tipo de error.
- toString()
 - Función: Devuelve una representación textual del error, útil para mostrar en consola.
 - Formato: "Error{Error encontrado en la línea - X, Lexema - 'Y', token - Z}"
- Getters y Setters.
- getLN(), getLex(), getToken(): Obtienen los valores de los atributos.
- setLN(int), setLex(String), setToken(String): Permiten modificar los valores de los atributos.

Notas de lógica interna:

- Esta clase es utilizada por Scanner para registrar errores léxicos durante el análisis.
- Permite una trazabilidad clara de los errores, indicando exactamente qué lexema falló y por qué.
- Su diseño es simple pero efectivo para diagnóstico y depuración del sistema.

5. Clase Sintactico:

Propósito: La clase Sintactico se encarga de realizar el análisis sintáctico y semántico de los tokens generados por el analizador léxico. Su objetivo es validar la estructura de las operaciones, verificar la semántica matemática y calcular los resultados de operaciones válidas.

Atributos:

- ArrayList<Token> filtro1: Tokens válidos tras el filtrado léxico.
- ArrayList<ArrayList<Token>> filtro2: Operaciones válidas tras análisis sintáctico.
- ArrayList<AES> erroresSintacticos: Lista de errores sintácticos y semánticos.
- HashMap<String, Float> operacionesResultados: Mapa de expresiones válidas y sus resultados.
- ArrayList<ArrayList<Token>> operacionesValidas: Lista de operaciones que pasaron todos los análisis.
- HashSet<Integer> lineasConErroresLexicos: Líneas que contienen errores léxicos.

Métodos principales:

- filtrarTokens(ArrayList<Token>, ArrayList<AER>)
 - Función: Orquesta el análisis en tres fases: léxico, sintáctico y semántico.
- verificarLexico(ArrayList<Token>)
 - Función: Filtra tokens válidos y registra errores si hay operadores inválidos.
- verificarSintaxis(ArrayList<Token>)
 - Función: Usa pilas para validar la estructura de operaciones anidadas y operandos.
 - Lógica: Verifica apertura/cierre de operaciones, cantidad de operandos, y etiquetas correctas.
- verificarSemantica(ArrayList<ArrayList<Token>>)
 - Función: Construye expresiones matemáticas y calcula sus resultados.
 - Manejo de errores: Detecta errores como división por cero, raíz de número negativo, etc.
- construirExpresion(ArrayList<Token>)
 - Función: Convierte una operación en una expresión tipo sum(2,3).
- calcularExpresion(String)
 - Función: Evalúa la expresión construida y devuelve el resultado.
- ejecutarOperacion(String, ArrayList<Float>)
 - Función: Ejecuta la operación matemática según el operador (sum, div, pow, etc.).
- registrarErrorSintactico(...) / registrarErrorSemantico(...)
 - Función: Añade errores a la lista con detalles de línea, tipo y contexto.
- imprimirErrores() / imprimirResultados()
 - Función: Muestra en consola los errores encontrados y los resultados calculados.

Notas de la lógica interna:

- Utiliza pilas para manejar operaciones anidadas y validar estructura.
- Verifica que los números estén correctamente etiquetados (<Numero>, <P>, <R>).
- Las operaciones como RAIZ y POTENCIA requieren exactamente un operando.
- Las operaciones como SUMA, RESTA, DIVISION, etc., requieren al menos dos operandos.

- El sistema detecta errores semánticos matemáticos como:
 - División por cero.
 - Raíz par de número negativo.
 - Inverso de cero.
 - Módulo por cero.
- Las expresiones se construyen en formato funcional (sum(2,3)), lo que facilita su evaluación.

6. Clase AES:

Propósito: La clase AES (Almacena Error Sintáctico/Semántico) encapsula la información de errores detectados durante el análisis sintáctico o semántico. Permite registrar, visualizar y gestionar errores con contexto detallado.

Atributos:

- int lineaError: Línea del archivo donde ocurrió el error.
- String contenido: Lexema o elemento que causó el error.
- String tipo: Tipo de error (e.g., "Error de sintaxis", "Error semántico").
- String contexto: Descripción adicional del error (opcional).

Métodos principales:

- AES(int linea, String contenido, String tipo)
 - Constructor: Inicializa un error sin contexto adicional.
- AES(int linea, String contenido, String tipo, String contexto)
 - Constructor: Inicializa un error con contexto detallado.
- toString()
 - Función: Devuelve una representación legible del error.
 - Formato: [Línea X] Tipo: 'Contenido' → Contexto
- Getters y Setters
 - Métodos para acceder y modificar los atributos: getLinea(), getContenido(), getTipo(), getContexto() y sus respectivos set.

Notas de la lógica interna.

- Se utiliza en la clase Sintactico para registrar errores durante el análisis.
- Permite diferenciar entre errores sintácticos y semánticos, y agregar contexto útil para depuración.
- Su diseño facilita la visualización clara y estructurada de los errores en consola.

7. Clase GeneradorArbol:

Propósito: La clase GeneradorArbol genera una representación gráfica en formato PNG de la jerarquía de operaciones aritméticas válidas. Cada operación se convierte en un árbol visual que muestra su estructura sintáctica y semántica.

Atributos:

- ARCHIVO_SALIDA: Nombre del archivo de imagen generado (ArbolJerarquia.png).
- Dimensiones y estilos:
 - ANCHO_NODO, ALTO_NODO: Tamaño de cada nodo.
 - ESPACIO_HORIZONTAL, ESPACIO_VERTICAL: Espaciado entre nodos.
 - PADDING_HORIZONTAL, PADDING_VERTICAL: Márgenes de la imagen.
 - COLOR_OPERACION, COLOR_NUMERO, COLOR_ESPECIAL: Colores por tipo de nodo.
 - TAMAÑO_FUENTE_*: Tamaño de fuente por tipo de nodo.

Métodos principales:

- generarArbolJerarquia(ArrayList<ArrayList<Token>> operacionesValidas)
 - Función: Genera y guarda la imagen del árbol de jerarquía para todas las operaciones válidas.
 - Lógica:
 - Construye árboles individuales con construirArbolOperacion().
 - Calcula dimensiones con calcularAnchosReales() y calcularAltoArbol().
 - Dibuja cada árbol con dibujarArbolOptimizado().
- construirArbolOperacion(ArrayList<Token> tokens)
 - Función: Construye el árbol raíz de una operación a partir de sus tokens.
 - Lógica: Busca el token de apertura de operación y llama a construirSubArbol().
- construirSubArbol(NodoArbol nodoActual, ArrayList<Token> tokens, int indice)
 - Función: Construye recursivamente los subárboles de una operación.
 - Lógica: Añade hijos según el tipo de token (operación, número, potencia, raíz).
- calcularAnchosReales(NodoArbol nodo)
 - Función: Calcula el ancho necesario para cada nodo y sus hijos.
- calcularAltoArbol(NodoArbol nodo)
 - Función: Calcula la altura del árbol en niveles.
- dibujarArbolOptimizado(Graphics2D g2d, NodoArbol nodo, int x, int y)
 - Función: Dibuja el árbol completo en la imagen, posicionando nodos y líneas.
- dibujarNodo(Graphics2D g2d, NodoArbol nodo, int x, int y)
 - Función: Dibuja un nodo individual con estilo visual según su tipo.

Notas de la lógica interna:

- Utiliza una clase interna NodoArbol para representar cada nodo del árbol.
- Cada nodo tiene:
 - valor: contenido textual.
 - tipo: operación, número o especial.
 - hijos: lista de nodos hijos.
 - anchoCalculado: usado para posicionamiento gráfico.
- El sistema distingue visualmente entre operaciones, números y elementos especiales (EXPONENTE, INDICE).
- Se optimiza el dibujo para evitar solapamientos y mejorar legibilidad.
- El resultado es una imagen clara y jerárquica que facilita la comprensión estructural de las operaciones analizadas.

8. Clase GeneradorHTML:

Propósito: La clase GeneradorHTML se encarga de generar archivos HTML que presentan los resultados del análisis léxico, sintáctico y semántico. Produce dos archivos:

- RESULTADOS_EQUIPO#3.html: muestra operaciones válidas y sus resultados.
- ERRORES_EQUIPO#3.html: muestra errores léxicos, sintácticos y semánticos.

Atributos:

- ARCHIVO_RESULTADOS: nombre del archivo HTML para resultados.
- ARCHIVO_ERRORES: nombre del archivo HTML para errores.

Métodos principales:

- generarArchivoResultados(ArrayList<ArrayList<Token>> operacionesValidas, HashMap<String, Float> resultados)
 - Función: Crea un archivo HTML con una tabla de operaciones válidas y sus resultados.
 - Contenido: Fecha, total de operaciones, tabla con número, expresión, resultado y línea.
- generarArchivoErrores(ArrayList<AER> erroresLexicos, ArrayList<AES> erroresSintacticos)
 - Función: Crea un archivo HTML con dos secciones:
 - Errores léxicos: línea, lexema, descripción.
 - Errores sintácticos/semánticos: línea, tipo, contenido, contexto.
- construirExpresionLegible(ArrayList<Token> operacion)
 - Función: Convierte una operación en una expresión legible como SUMA(2, 3).
- buscarResultado(HashMap<String, Float> resultados, ArrayList<Token> operacion)

- Función: Reconstruye la expresión interna (sum(2,3)) para buscar su resultado en el mapa.
- `formatearResultado(Float resultado)`
 - Función: Formatea el resultado numérico:
 - Enteros sin decimales.
 - Decimales con hasta 4 cifras.
- `escaparHTML(String texto)`
 - Función: Escapa caracteres especiales para evitar errores de renderizado en HTML.
- `obtenerFechaActual()`
 - Función: Devuelve la fecha y hora actual en formato dd/MM/yyyy HH:mm:ss.
- `escribirArchivo(String nombreArchivo, String contenido)`
 - Función: Escribe el contenido HTML en el archivo especificado.

Notas de la lógica interna:

- Usa `StringBuilder` para construir el HTML de forma eficiente.
- Separa claramente los errores léxicos de los sintácticos/semánticos.
- Las expresiones se reconstruyen desde los tokens para mostrar resultados coherentes.
- Los estilos CSS están definidos en métodos separados (`generarEstilosResultados`, `generarEstilosErrores`), aunque en esta versión parecen vacíos o no implementados.
- El sistema escapa caracteres HTML para evitar conflictos de renderizado.

9. Clase InterfazGrafica:

Propósito: La clase `InterfazGrafica` implementa la interfaz gráfica de usuario (GUI) del sistema de análisis de operaciones aritméticas. Permite al usuario cargar, editar, analizar archivos .txt, y visualizar los resultados y errores mediante reportes HTML y gráficos PNG.

Atributos:

- Componentes de interfaz:
 - `JTextArea areaTexto`: Área principal para editar el texto fuente.
 - `File archivoActual`: Archivo actualmente abierto o editado.
- Módulos de análisis:
 - `Scanner scanner`: Analizador léxico.
 - `Sintactico analizadorSintactico`: Analizador sintáctico y semántico.
- Generadores de reportes:
 - `GeneradorHTML generadorHTML`: Genera reportes HTML.
 - `GeneradorArbol generadorArbol`: Genera imagen PNG del árbol jerárquico.
- Constantes de interfaz:
 - `TITULO_APLICACION`, `ANCHO_VENTANA`, `ALTO_VENTANA`: Configuración visual.

Métodos principales:

- `InterfazGrafica()`
 - Constructor que inicializa componentes, configura la ventana y muestra la interfaz.
- `inicializarComponentes()`
 - Instancia los módulos de análisis y generadores de reportes.
- `configurarVentana()`
 - Define propiedades de la ventana principal (tamaño, título, icono, layout).
- `crearMenuBar()`
 - Crea la barra de menú con opciones de archivo, análisis y ayuda.
- `crearAreaTexto()`
 - Configura el área de texto con numeración de líneas y estilo monoespaciado.
- `abrirArchivo()`, `guardarArchivo()`, `guardarArchivoComo()`
 - Permiten abrir y guardar archivos .txt desde el sistema de archivos.
- `analizarTexto()`
 - Ejecuta el análisis léxico, sintáctico y semántico sobre el contenido del área de texto.
 - Muestra un diálogo de progreso mientras se realiza el análisis.
 - Genera reportes HTML y el árbol gráfico.
- `generarArchivosReporte()`
 - Llama a los generadores para crear:
 - `RESULTADOS_EQUIPO#3.html`
 - `ERRORES_EQUIPO#3.html`
 - `ArbolJerarquia.png` (si hay operaciones válidas)
- `mostrarResultadosAnalisis()`
 - Muestra un resumen con:
 - Total de tokens
 - Errores léxicos
 - Errores sintácticos/semánticos
 - Operaciones válidas
 - Archivos generados
- `abrirManual(String nombreArchivo)`
 - Abre los manuales PDF desde el sistema operativo.
- `mostrarError(String mensaje)`
 - Muestra un diálogo de error.
- `salirAplicacion()`
 - Cierra la aplicación con confirmación.

Notas de la lógica interna:

- Usa `SwingWorker` para ejecutar el análisis en segundo plano sin congelar la interfaz.
- El análisis se realiza sobre un archivo temporal generado desde el área de texto.

- La interfaz está diseñada para ser intuitiva y funcional, con accesos rápidos y retroalimentación visual.
- Los reportes generados permiten auditoría completa del análisis realizado.